

Overview and Scope

This will be a Proof-of-Concept (PoC) to show basic Tactical Microgrid operation. As such, we'll just do a minimalistic exchange of information to provide a basic handshake and energizing of the generator to the microgrid. It will be made up of two applications, a Source Device (a.k.a. Generator) and enough of a Microgrid Controller to drive the Source Device. As a Proof-of-Concept (PoC) example, we won't implement much of the detail. Example - MC selection state machine and detail Device Capabilities e.g. Power etc. Below is the part of Table B.2 that the PoC will implement

Topic	Device	Controller	QoS Profile (A.8.5.4)	Pub Trigger
TOPIC_DEVICE_INFO	Pub/Sub* (F-MC)	Pub/Sub*	Publish Last (Durable)	At Start
TOPIC_HEARTBEAT	Pub/Sub* (F-MC)	Pub/Sub*	Periodic Medium (1/sec), Best Effort	Periodic
TOPIC_ACTIVE_MICROGRID_CONTROLLER_STATE	Pub	Sub	Publish Last	As Needed
TOPIC_AUTHORIZATION_TO_ENERGEIZE_REPLY	Sub (F)	Pub	Response (Reliable)	As Needed
TOPIC_AUTHORIZATION_TO_ENERGEIZE_REQUEST	Pub	Sub	Command (Reliable)	As Needed
TOPIC_AUTHORIZATION_TO_ENERGEIZE_RESULT	Pub	Sub	Response (Reliable)	Within .1s of Request
TOPIC_ENERGY_START_STOP_STATE	Pub	Sub	Publish Last (Durable)	Start, On Change
TOPIC_ENERGY_START_STOP_REQUEST	Sub (F)	Pub	Command (Reliable)	As Needed
TOPIC_REPLY	Pub	Sub	Reply	Within .1s of Request

* - Used for MC Selection, The PoC has only one MC and won't run the MC Selection process. The default receive handlers will print the topic

(F) - Filter for topics with only my DeviceId.

Controller(s) will not filter, but only act if they received a TOPIC_ACTIVE_MICROGRID_CONTROLLER_STATE¹ with their ID

This PoC will be implemented in Python and Traditional C++, using "Command/Response" wrapper classes. These wrapper classes do the DDS "heavy lifting" (refer to Command and Control wrapper classes and their documentation in [GitHub](#) regarding how these work) The python version of the application uses RTI's System Designer with the TMS IDL files to standup

¹ Authors note: The controllers themselves could determine who is master by running the same Controller Selection Algorithm defined in section A7.7 of the TMS specification - this would abstract and present a single logical controller to the device while lessening the burden (no need to for the Device to receive Heartbeat , DeviceInfo, or send ActiveMicrogridControllerState.)

all entities via XML Application Create along with RTI's Dynamic Data feature. The Traditional c++ will use compiled types and programatic entity creation.

Content Filters

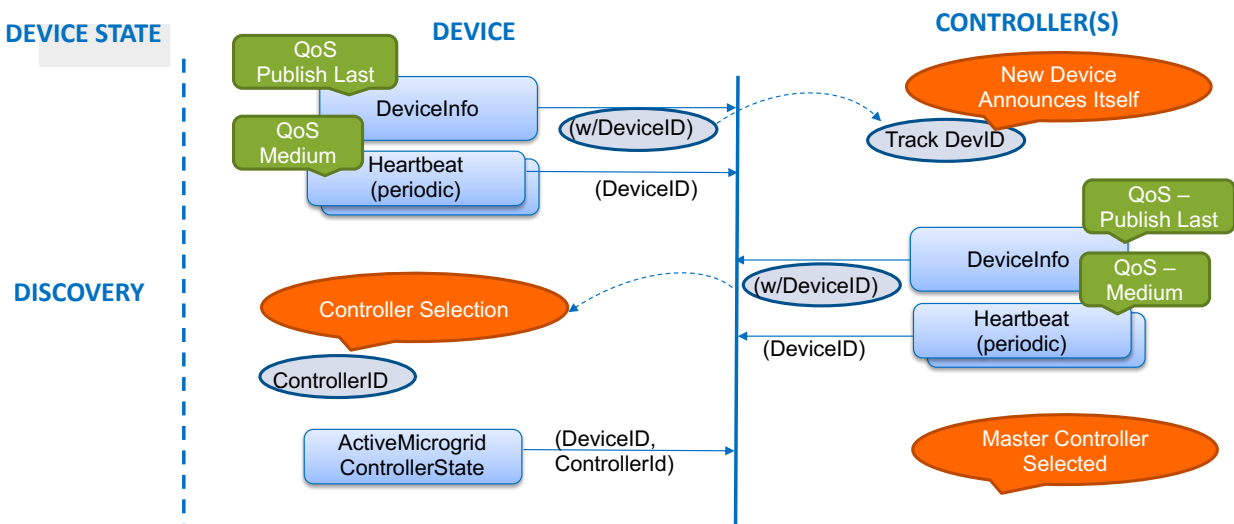
Content filters will be implemented per the table above that are designated with an (F). This include only device reader topics with the exception of `TOPIC_DEVICE_INFO` and `TOPIC_HEARTBEAT`.

Ignore Data Writer

For topics that are both published and received by a participant, a `participant.ignore_datawriter (writer_handle)` is called. This ensures the application only gets samples sent from other applications and not the samples the application itself sends.

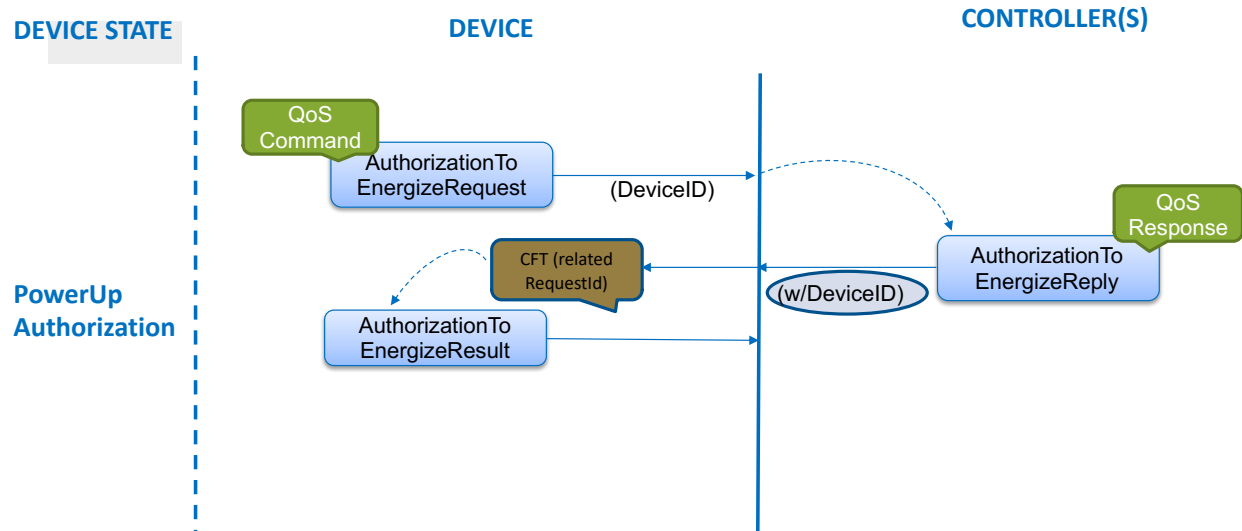
PoC Message Exchange

Discovery² and Selection

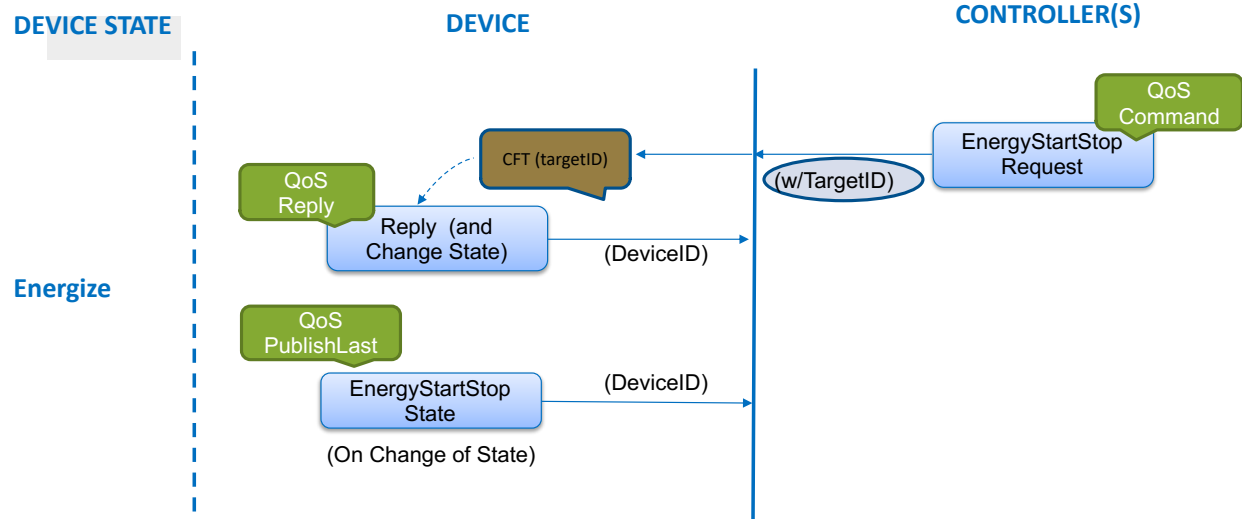


² Note: Device and Controller independently start discovery and are not order dependent. Controller Selection is dependent upon Controller `DeviceInfo` and `Heartbeat`.

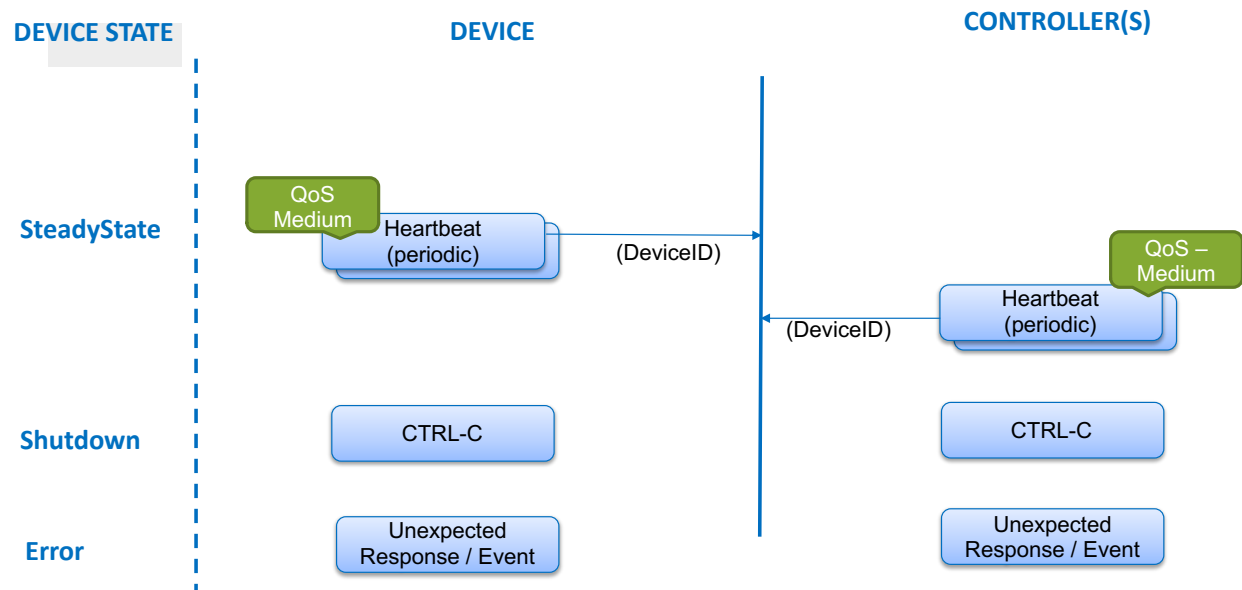
Power Up Authorization



Energize



Steady State, Shutdown, Error



Interesting DDS example code:

1. Heartbeat and Device Info readers ignore their own written data
 - a. See `topics.h` `HeartbeatXX_Rdr` topics constructors.
`participant->ignor_publication()`
2. Content filter on device topics, so they only get commands are replies set to the DeviceID (my device). This allows the application to not concern itself about getting the same commands or replies intended for a different device.
 - a. See `device.cxx` or `device.py` application files
3. DeviceInfo Type Object larger than default (2048 bytes) blocks discovery
 - a. Used QoS profile "LargeTopicParticipantQoS" as defined in XML QoS Libs when creating participant
4. Nested Device Info Sequence writing
 - a. See `topics.h` `DeviceInfoGD_Wtr` topics
5. Optional Data Members Writing and Reading
 - a. Writing Optional Members need to allocate Memory - see `Topic_T.h` `writer create_data()`
 - b. Readers of Optional Members sent from a writer need to first validate if they exist. This is done by checking the optional member pointer != Null. An example of this is provided in `topics.h` `AMCStateMC_Rdr` class

Appendix E - Profile

Profiled on a MacBookPro (2.4GHzQuadCore I5) - 2020 / Catalina 10.15.4

Threads: 17

Memory: 13.5MB

CPU - .1% idle (noManager) .2% operating.

Wake's 0 idle , 27 Operating